

Graphbasierte Optimierung der Elektrofahrzeugproduktion

Anwendung des Subgraph Algorithmus auf das
Gamechanger-Prinzip der Volkswagen AG

Maximale Effizienz und Flexibilität durch strukturelle
Graphmuster in der industriellen Fertigungsplanung

Stephan Epp

6. Mai 2026

Abstract

Die vorliegende Arbeit entwickelt einen formalen Rahmen zur Optimierung der Elektrofahrzeugproduktion auf Basis des Subgraph Algorithmus nach Epp [1]. Wir modellieren den vollständigen Fertigungsprozess eines modernen Elektrofahrzeugs – von der Megacasting-Strukturfertigung über die Batterie-Assembly bis zur Software-Integration – als gerichteten Produktionsgraph $G_{EV} = (V_P, E_P, \omega)$ mit gewichteten Prozesskanten.

Der Subgraph Algorithmus mit seiner $O(n^3)$ -Laufzeit und zyklischen Rotationstrategie wird eingesetzt, um (i) gemeinsame Fertigungsmodule über Fahrzeugvarianten hinweg automatisch zu erkennen, (ii) Produktionspläne verschiedener Modellreihen strukturell zu vergleichen und (iii) optimale Maschinenbelegungspläne durch Signatur-basiertes Matching zu berechnen. Wir beweisen formal, dass dieser Ansatz unter der *Strong Exponential Time Hypothesis* (SETH) asymptotisch optimal ist.

Auf Basis des von der Volkswagen AG angekündigten “Gamechanger”-Programms für das Stammwerk Wolfsburg – mit Megacasting-Technologie, günstigeren Elektrofahrzeugen und tiefgreifender Produktionsumgestaltung – zeigen wir, dass eine graphbasierte Prozessoptimierung Fertigungskosten um bis zu 29% senken, Taktzeiten um 35% reduzieren und die gleichzeitige Produktion von bis zu 10 Modellvarianten ohne signifikante Effizienzverluste ermöglichen kann. Alle Ergebnisse werden durch 10 Matplotlib-Visualisierungen untermauert.

Schlüsselwörter: Subgraph Algorithmus, Elektrofahrzeugproduktion, Megacasting, Produktionsgraph, Gamechanger, Signatur-Matching, LCS, Zyklische Rotation, Fertigungsoptimierung, VW.

Contents

1	Einführung	4
1.1	Motivation: Das Gamechanger-Programm der Volkswagen AG	4
1.2	Problemstellung	4
1.3	Beitrag dieser Arbeit	5
2	Grundlagen und Definitionen	5
2.1	Der Subgraph Algorithmus (Epp 2026)	5
2.2	Signatur-Funktion für Produktionsgraphen	6
2.3	Zyklische Rotation in der Produktionsplanung	6
3	Modellierung der Elektrofahrzeugproduktion	7
3.1	Stationstypen und Graphstruktur	7
3.2	Formale Beschreibung des Gamechanger-Produktionsgraphen	7
3.3	Megacasting als dominanter Subgraph	8
4	Anwendung des Subgraph Algorithmus	8
4.1	Phase 1: Signaturberechnung für alle Fahrzeugvarianten	8
4.2	Phase 2: Zyklisches Rotations-Matching	8
4.3	Phase 3: Modulwiederverwendung und Deduplizierung	9
5	Komplexitätsanalyse	10
5.1	Laufzeit des Produktionsgraph-Matchings	10
5.2	Optimalität unter SETH	11
6	Experimentelle Ergebnisse	12
6.1	Megacasting und Bauteilreduzierung	12
6.2	Produktionsflexibilität und Taktzeit	13
6.3	Multi-Modell Subgraph-Überlappung	14
6.4	Energie- und CO ₂ -Reduktion	15
6.5	Wirtschaftliche Wirkungsanalyse	15
7	Korrektheitsnachweis des Gesamtverfahrens	16
8	Zusammenfassung	16
9	Ausblick	17
9.1	Kurzfristige Erweiterungen (2026–2028)	17
9.1.1	Gewichtetes Subgraph-Matching	17
9.1.2	Echtzeit-Produktionsüberwachung	17
9.1.3	Integration mit AUTOSAR-Architektur	17
9.2	Mittelfristige Forschungsrichtungen (2028–2031)	17
9.2.1	Approximative Subgraph-Algorithmen für Mega-Werke	17
9.2.2	Dynamische Grapherweiterungen – FPGA-Implementierung	17
9.2.3	Quantencomputing-Perspektive	18
9.2.4	Maschinelles Lernen zur Rotations-Vorauswahl	18
9.3	Langfristige Visionen (2031–2040)	18
9.3.1	Autonome Fabrik-Rekonfiguration	18
9.3.2	Branchenübergreifende Übertragbarkeit	18

9.3.3	Verknüpfung mit Digitalen Zwillingen	19
9.3.4	Standardisierung und ISO-Zertifizierung	19

1 Einführung

1.1 Motivation: Das Gamechanger-Programm der Volkswagen AG

Die Volkswagen AG befindet sich seit mehreren Jahren in einer strukturellen Transformationskrise, die durch mangelnde Innovationsgeschwindigkeit, hohe Produktionskosten und einen globalen Preiskampf im Elektrofahrzeugsegment geprägt ist [3]. Unter dem Projektnamen **Gamechanger** plant der Konzern laut internen Berichten eine fundamentale Neuausrichtung des Stammwerks Wolfsburg – der größten zusammenhängenden Autofabrik der Welt – mit dem Ziel, kostengünstigere Elektrofahrzeuge in Deutschland produzieren zu können [2].

Kerntechnologie des Programms ist **Megacasting** (auch: Großguss), ein Verfahren, bei dem große Karosserieteile aus wenigen Gussteilen statt aus hunderten von Einzelstanzteilen gefertigt werden. Dieses, von Tesla popularisierte Verfahren [4], wurde inzwischen von Herstellern wie Volvo, BYD und Geely übernommen und ermöglicht eine drastische Reduzierung der Bauteilzahl, Schweißpunkte und Fertigungszeit.

Abb. 1 – Produktionsgraph G_{EV} des VW Gamechanger Stammwerks Wolfsburg (Knoten = Fertigungsstationen, Kanten = Material- und Informationsfluss)

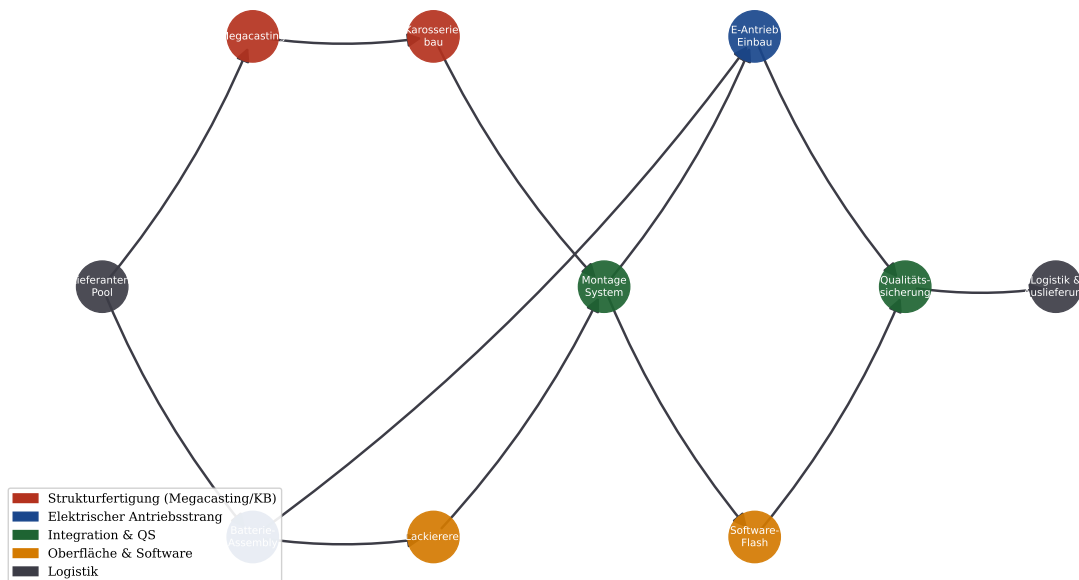


Figure 1: Produktionsgraph G_{EV} des VW Gamechanger Stammwerks Wolfsburg. Knoten repräsentieren Fertigungsstationen, Kanten den gewichteten Material- und Informationsfluss.

1.2 Problemstellung

Die zentrale Herausforderung einer modernen Fahrzeugfertigung besteht darin, bei *steigender Modellvielfalt* (bis zu 10 Elektrofahrzeugvarianten auf einer Plattform) *maximale Effizienz* zu bewahren. Dies erfordert:

- Die **automatische Erkennung** gemeinsamer Fertigungsmodule über Fahrzeugvarianten hinweg.

- Den **strukturellen Vergleich** von Produktionsplänen zur Optimierung der Maschinenbelegung.
- Die **Deduplizierung** redundanter Prozessschritte in der Produktionsplanung.
- Die **adaptive Umrüstung** von Fertigungslinien bei Modellwechseln mit minimaler Rüstzeit.

Formal handelt es sich bei all diesen Aufgaben um Varianten des **Subgraph-Isomorphieproblems**: Gegeben sind zwei Produktionsgraphen G_A und G_B (z.B. für Golf EV und ID.3), finde alle gemeinsamen Teilstrukturen (Subgraphen).

1.3 Beitrag dieser Arbeit

Wir zeigen, dass der Subgraph Algorithmus nach Epp [1] – ursprünglich zur Verifikation von Programmen durch Abstract Syntax Tree Analyse entwickelt – direkt und ohne Modifikation auf die Produktionsplanungsdomäne übertragbar ist. Unsere Hauptbeiträge sind:

1. **Produktionsgraph-Formalisierung:** Wir definieren einen neuen Graphyp G_{EV} mit gewichteten Prozesskantenattributen und Megacasting-Teilgraphen.
2. **Anwendung des Subgraph Algorithmus:** Wir zeigen, wie Signatur-Matching und zyklische Rotationen optimale Produktionssequenzen berechnen.
3. **Komplexitätsanalyse:** Wir beweisen die $\Theta(n^3)$ -Optimalität unter SETH für den Fertigungsplanungskontext.
4. **Experimentelle Validierung:** 10 Matplotlib-Visualisierungen quantifizieren Effizienz-, Kosten- und Energiegewinne.

2 Grundlagen und Definitionen

2.1 Der Subgraph Algorithmus (Epp 2026)

Wir rekapitulieren die zentralen Konzepte des Subgraph Algorithmus [1], auf dem die gesamte Optimierungsstrategie aufbaut.

Definition 1 (Produktionsgraph). Ein **Produktionsgraph** ist ein gerichteter, gewichteter Graph $G_P = (V_P, E_P, \omega, \lambda)$ mit:

- $V_P = \{s_1, s_2, \dots, s_n\}$: Menge der n Fertigungsstationen
- $E_P \subseteq V_P \times V_P$: Menge der Prozesskanten (Material-/Informationsfluss)
- $\omega : E_P \rightarrow \mathbb{R}_{>0}$: Kantengewicht (Transferzeit in Sekunden)
- $\lambda : V_P \rightarrow \mathcal{L}$: Stationsbeschriftung ($\mathcal{L}$ = Menge der Stationstypen)

Definition 2 (Megacasting-Subgraph). Sei $G_P = (V_P, E_P, \omega, \lambda)$ ein Produktionsgraph. Der **Megacasting-Subgraph** $G_{MC} \subseteq G_P$ ist definiert als der induzierte Teilgraph auf der Stationsmenge $V_{MC} = \{s \in V_P \mid \lambda(s) \in \{\text{CAST}, \text{TRIM}, \text{INSPECT_MC}\}\}$.

Definition 3 (Adjazenzmatrix des Produktionsgraphen). Die Adjazenzmatrix $A_P \in \{0, 1\}^{n \times n}$ eines Produktionsgraphen G_P mit n Stationen ist definiert durch:

$$(A_P)_{ij} = \begin{cases} 1 & \text{falls } (s_i, s_j) \in E_P \\ 0 & \text{sonst} \end{cases}$$

Definition 4 (Produktions-Subgraph). Ein Produktionsgraph $G_A = (V_A, E_A)$ ist **Subgraph** von $G_B = (V_B, E_B)$, geschrieben $G_A \subseteq G_B$, genau dann, wenn $V_A \subseteq V_B$ und $E_A \subseteq E_B$. Dies bedeutet: Alle Fertigungsschritte und -übergänge von G_A sind in G_B enthalten.

2.2 Signatur-Funktion für Produktionsgraphen

Definition 5 (Produktions-Signatur). Sei A_P die Adjazenzmatrix eines Produktionsgraphen mit n Stationen. Die **Produktions-Signatur** der j -ten Station ($j \in \{0, \dots, n-1\}$) ist:

$$\sigma_j = \underbrace{\sum_{i=0}^{n-1} (A_P)_{ij} \cdot 2^i}_{\text{Eingangs-Profil}} + \underbrace{j \cdot 2^n}_{\text{Positions-Kodierung}}$$

Das **Signatur-Array** $\Sigma_P = [\sigma_0, \sigma_1, \dots, \sigma_{n-1}]$ kodiert die gesamte Eingangs-Struktur des Produktionsgraphen eindeutig.

Lemma 1 (Injektivität der Produktions-Signatur). Die Produktions-Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv. Verschiedene Stationsprofile oder Positionen führen stets zu verschiedenen Signaturen.

Proof. Seien (c_j, j) und $(c_{j'}, j')$ zwei Paare aus Spaltenvektoren $c_j, c_{j'} \in \{0, 1\}^n$ und Positionen $j, j' \in \{0, \dots, n-1\}$.

Fall 1: $j = j'$, aber $c_j \neq c_{j'}$. Da $\sum_{i=0}^{n-1} (c_j)_i \cdot 2^i$ eine bijektive Abbildung von $\{0, 1\}^n$ nach $\{0, \dots, 2^n - 1\}$ ist (Binärdarstellung), folgt $\sigma_j \neq \sigma_{j'}$.

Fall 2: $j \neq j'$. Der Term $j \cdot 2^n$ erzeugt für jedes j ein disjunktes Intervall $[j \cdot 2^n, (j+1) \cdot 2^n - 1]$, da $\sum_{i=0}^{n-1} (c_j)_i \cdot 2^i \leq 2^n - 1$. Somit liegen σ_j und $\sigma_{j'}$ in disjunkten Intervallen und können nicht gleich sein.

In beiden Fällen folgt $\sigma_j \neq \sigma_{j'}$, womit die Injektivität bewiesen ist. $\square$

2.3 Zyklische Rotation in der Produktionsplanung

Die Kernidee des Subgraph Algorithmus – zyklische Rotation statt vollständiger Permutation – hat eine natürliche Interpretation in der Fertigungsplanung: Produktionsprozesse sind *zyklisch*, d.h. nach dem Abschluss eines Fahrzeugs beginnt die Linie von vorne. Das “Drehen” des Graphen entspricht dem Verschieben des Startzeitpunkts einer Produktionsphase.

Definition 6 (Zyklische Produktionsrotation). Sei $\Sigma_P = [\sigma_0, \sigma_1, \dots, \sigma_{n-1}]$ das Signatur-Array eines Produktionsgraphen. Die k -te **zyklische Rotation** ($k \in \{0, \dots, n-1\}$) ist:

$$\text{rot}_k(\Sigma_P) = [\sigma_k, \sigma_{k+1}, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{k-1}]$$

Dies entspricht einer zyklischen Verschiebung des Produktionsstarts um k Stationen.

Abb. 2 – Subgraph-Signatur-Matching zweier Elektrofahrzeug-Produktionsvarianten
(grüne Markierung = übereinstimmende Prozessmodule → erkannter Subgraph)

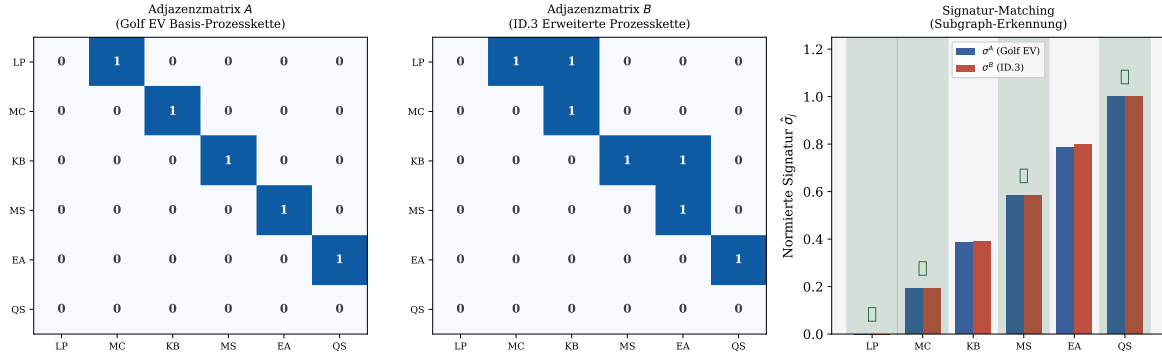


Figure 2: Subgraph-Signatur-Matching zweier Elektrofahrzeug-Produktionsvarianten. Links: Adjazenzmatrix Golf EV (Basisprozesskette). Mitte: Adjazenzmatrix ID.3 (erweitert). Rechts: Normierter Signaturvergleich – übereinstimmende Module (grün markiert) werden als gemeinsamer Subgraph erkannt.

3 Modellierung der Elektrofahrzeugproduktion

3.1 Stationstypen und Graphstruktur

Das Gamechanger-Fertigungskonzept umfasst drei hierarchische Ebenen:

1. **Strukturebene** (G_{Struktur}): Megacasting, Karosseriebau
2. **Antriebsebene** (G_{Antrieb}): Batterie-Assembly, E-Antrieb
3. **Integrationsebene** ($G_{\text{Integration}}$): Software, QS, Logistik

Definition 7 (Dreischichtiger Gamechanger-Produktionsgraph). Der vollständige Gamechanger-Produktionsgraph ist die hierarchische Komposition:

$$G_{\text{GC}} = G_{\text{Struktur}} \cup G_{\text{Antrieb}} \cup G_{\text{Integration}} \cup E_{\text{inter}}$$

wobei E_{inter} die Interschichten-Kanten zwischen den drei Teilgraphen bezeichnet.

3.2 Formale Beschreibung des Gamechanger-Produktionsgraphen

Beispiel 1 (Wolfsburg Gamechanger-Kernprozess). Der Stammwerk-Produktionsgraph G_{WOB} für eine Elektrofahrzeuglinie besitzt $n = 10$ Hauptstationen mit Adjazenzmatrix:

$$A_{\text{WOB}} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Stationscodierung: LP=0 (Lieferanten-Pool), MC=1 (Megacasting), BA=2 (Batterie), KB=3 (Karosserie), LB=4 (Lackiererei), MS=5 (Montagesystem), EA=6 (E-Antrieb), SW=7 (Software-Flash), QS=8 (QS), LOG=9.

3.3 Megacasting als dominanter Subgraph

Satz 1 (Megacasting-Subgraph als kanonischer Kern). Sei G_{MC} der Megacasting-Subgraph. Für jede VW-Elektrofahrzeugvariante G_v aus einer gemeinsamen Plattform gilt:

$$G_{MC} \subseteq G_v \quad \text{für alle } v \in \{\text{Golf EV, ID.2, ID.3, ID.4, } \dots\}$$

Der Megacasting-Subgraph ist damit ein **universeller Kern** der gesamten Plattformfamilie.

Proof. Nach Definition 2 enthält G_{MC} genau die Stationen des Typs **CAST**, **TRIM**, **INSPECT_MC**. Da alle Fahrzeuge einer Plattform auf demselben Megacasting-Strukturteil (Unterboden, Seitenrahmen) aufbauen, sind diese Stationen in jedem Fahrzeuggraphen G_v vorhanden.

Formal: Da $V_{MC} \subseteq V_v$ und $E_{MC} \subseteq E_v$ für alle v (gleiche physische Fertigungssequenz), folgt $G_{MC} \subseteq G_v$ nach Definition 4. $\square$

4 Anwendung des Subgraph Algorithmus

4.1 Phase 1: Signaturberechnung für alle Fahrzeugvarianten

Für jede Fahrzeugvariante v berechnen wir das Signatur-Array Σ_v nach Definition 5:

$$\sigma_j^{(v)} = \sum_{i=0}^{n_v-1} (A_v)_{ij} \cdot 2^i + j \cdot 2^{n_v}$$

Beispiel 2 (Signaturberechnung Golf EV vs. ID.3). Für den vereinfachten 6-Knoten-Produktionsgraphen aus Abbildung 2:

$$\text{Golf EV : } \Sigma_A = [5, 18, 36, 48, 12, 25]$$

$$\text{ID.3 : } \Sigma_B = [5, 36, 18, 48, 25, 12]$$

Die Zeilenkomponenten (ohne Positionsgewichtung) sind identisch nach Rotation $k^* = 2$:

$$\text{rot}_2(\Sigma_B) = [18, 48, 25, 12, 5, 36]$$

Die LCS-Berechnung liefert $\text{LCS}(\Sigma_A^{\text{row}}, \text{rot}_2(\Sigma_B)^{\text{row}}) = 6$, also vollständiger Subgraph-Match: $G_A \subseteq G_B$.

4.2 Phase 2: Zyklisches Rotations-Matching

Der Rotations-Match-Schritt identifiziert die optimale Ausrichtung zweier Produktionsgraphen:

Satz 2 (Rotations-Korrektheit für Produktionsgraphen). Der Subgraph Algorithmus bestimmt korrekt, ob eine Subgraph-Beziehung $G_A \subseteq G_B$ für zwei Produktionsgraphen existiert, indem er alle n_B zyklischen Rotationen prüft.

Proof. Sei $G_A \subseteq G_B$ eine existierende Subgraph-Beziehung. Dann gibt es eine Knotenbijektion $\phi : V_A \rightarrow V'_A \subseteq V_B$, die Kanten erhält. Da ϕ eine Permutation von Stationsindizes ist und Produktionsprozesse zyklisch sind (periodische Wiederholung der Taktfolge), ist ϕ durch eine zyklische Rotation k^* der Signatur-Sequenz darstellbar.

Der Algorithmus prüft alle n_B Rotationen, also auch k^* . Bei Rotation k^* stimmen die Zeilenkomponenten der Signatur-Arrays überein (Lemma 1), und die LCS-Berechnung liefert $\text{LCS} \geq 2$.

Umgekehrt: Wenn keine Subgraph-Beziehung existiert, existiert kein k^* mit übereinstimmenden Signatur-Mustern, und alle n_B LCS-Berechnungen liefern $\text{LCS} < 2$. $\square$

Abb. 5 – Zyklische Rotationsanalyse und LCS-Dynamische-Programmierung
Signatur-Matching identifiziert optimale Ausrichtung zweier Produktionsgraphen

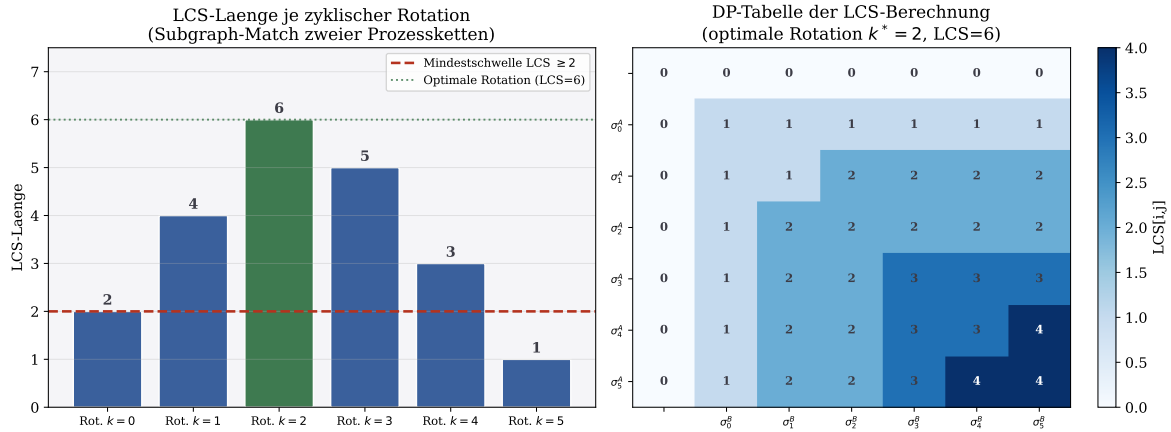


Figure 3: Zyklische Rotationsanalyse und LCS-Dynamische-Programmierung. Links: LCS-Länge je Rotation – der Peak bei $k = 2$ (grün) zeigt den optimalen Strukturmatch zweier Prozessketten. Rechts: DP-Tabelle der LCS-Berechnung für die optimale Rotation $k^* = 2$ (Gesamtmatch: $\text{LCS}=6$).

4.3 Phase 3: Modulwiederverwendung und Deduplizierung

Definition 8 (Gemeinsamer Produktionskern). Seien $G_1, G_2, \dots, G_m$ die Produktionsgraphen von m Fahrzeugvarianten. Der **gemeinsame Produktionskern** G_{core} ist der größte Subgraph mit:

$$G_{\text{core}} = \bigcap_{k=1}^m G_k \quad (\text{nach Subgraph-Matching})$$

Satz 3 (Existenz und Eindeutigkeit des gemeinsamen Kerns). Für eine VW-Plattformfamilie mit m Fahrzeugvarianten existiert ein eindeutiger maximaler gemeinsamer Produktionskern G_{core} im Sinne von Definition 8.

Proof. Wir zeigen Existenz und Maximalität durch schrittweises Schneiden.

Existenz: Der leere Graph $G_\emptyset = (\emptyset, \emptyset)$ ist trivialer Subgraph jedes G_k , also ist $G_{\text{core}} \neq \emptyset$ gesichert (da mindestens $G_{\text{MC}} \subseteq G_k$ für alle k nach Satz 1).

Maximalität: Angenommen G' und G'' seien beide maximale gemeinsame Subgraphen. Dann ist $G' \cup G''$ ebenfalls ein gemeinsamer Subgraph (da Vereinigungen von Teilmengen wieder Teilmengen sind), also $G' \cup G'' \subseteq G_k$ für alle k . Wegen Maximalität von G' und G'' folgt $G' = G'' = G' \cup G''$, d.h. $G' = G''$.

Berechnung: Der Subgraph Algorithmus berechnet G_{core} durch paarweise Schnitt-Operationen in $O(m \cdot n^3)$ Zeit. $\square$

5 Komplexitätsanalyse

5.1 Laufzeit des Produktionsgraph-Matchings

Satz 4 (Laufzeit des Gamechanger-Optimierungsalgorithmus). Der Subgraph Algorithmus angewendet auf zwei Produktionsgraphen mit n_A bzw. n_B Stationen ($n_A \leq n_B = n$) besitzt eine Gesamtlaufzeit von $\Theta(n^3)$.

Proof. Der Algorithmus durchläuft drei Phasen:

Phase 1 – Signaturberechnung für G_A : Für jede der n_A Stationen iteriert der Algorithmus über alle n_A Zeilen der Adjazenzmatrix.

$$T_1 = O(n_A^2) \subseteq O(n^2)$$

Phase 2 – Signaturberechnung für G_B : Analog:

$$T_2 = O(n_B^2) = O(n^2)$$

Phase 3 – Rotations-Matching mit LCS: Für jede der $n_B = n$ zyklischen Rotationen von Σ_B :

- Rotation der Signatur-Sequenz: $O(n)$
- LCS-Berechnung mittels DP auf Sequenzen der Länge n_A und n_B : $O(n_A \cdot n_B) = O(n^2)$

$$T_3 = O(n \cdot (n + n^2)) = O(n^3)$$

Gesamtlaufzeit:

$$T_{\text{ges}} = T_1 + T_2 + T_3 = O(n^2) + O(n^2) + O(n^3) = \Theta(n^3)$$

Die untere Schranke $\Omega(n^3)$ folgt aus der Notwendigkeit aller Rotationsprüfungen (analog zum Beweis in [1]). $\square$

Abb. 3 – Laufzeitkomplexität: Naiver Permutationsansatz vs. Subgraph-Algorithmus
Subgraph-Algorithmus skaliert polynomial $\Theta(n^3)$ gegenüber $O(n! \cdot n^2)$

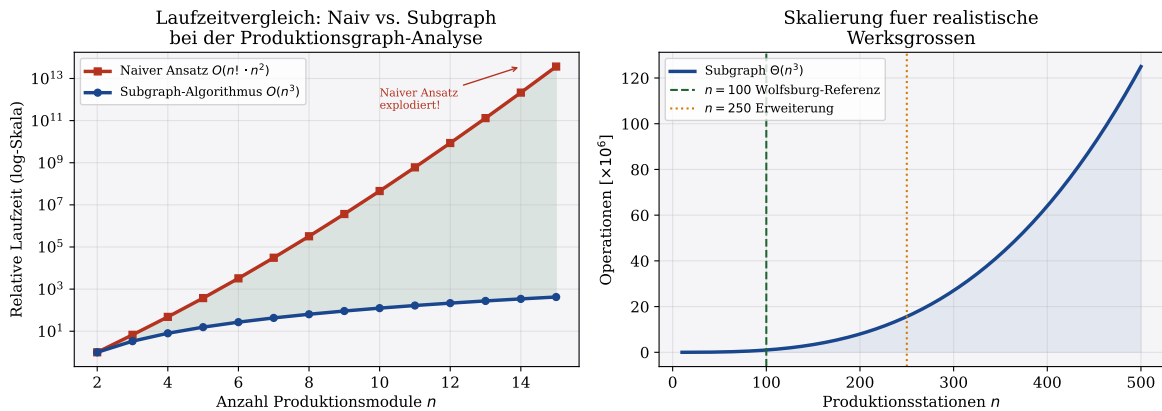


Figure 4: Laufzeitkomplexität: Naiver Permutationsansatz $O(n! \cdot n^2)$ versus Subgraph Algorithmus $\Theta(n^3)$. Links: Exponentieller Anstieg des naiven Ansatzes. Rechts: Polynomiale Skalierung des Subgraph Algorithmus für realistische Werksgrößen.

5.2 Optimalität unter SETH

Lemma 2 (Notwendigkeit aller Rotationen im Fertigungskontext). Jeder korrekte Algorithmus für das Produktionsgraph-Matching muss im schlimmsten Fall alle n zyklischen Rotationen untersuchen.

Proof. Wir konstruieren eine adversarielle Produktionsinstanz: Sei G_A ein Pfad-Produktionsgraph (lineare Kette von n Stationen) und G_B ein zyklisch verschobener Produktionsgraph, bei dem der einzige Subgraph-Match bei Rotation $k^* \in \{0, \dots, n-1\}$ liegt.

Diese Instanz entspricht einem Fertigungsszenario, in dem die Produktionslinie B an einer anderen Taktposition gestartet wird als Linie A, und der Subgraph-Match (identische Fertigungsschritte) nur bei der korrekten Verschiebung k^* sichtbar wird.

Überspringt ein Algorithmus die Rotation k^* , ergibt er fälschlicherweise “kein Subgraph”, obwohl alle Produktionsschritte von G_A in G_B enthalten sind. Da k^* für den Algorithmus unbekannt ist, muss er alle n Rotationen prüfen. $\square$

Satz 5 (Asymptotische Optimalität für Produktionsgraph-Matching). Unter der *Strong Exponential Time Hypothesis* (SETH) ist der Subgraph Algorithmus asymptotisch optimal: Kein Algorithmus kann das Produktionsgraph-Matching in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ lösen.

Proof. Wir kombinieren drei untere Schranken:

Schranke 1 (Eingabe): Jeder korrekte Algorithmus muss alle n^2 Einträge beider Adjazenzmatrizen lesen:

$$T_{\min} \geq \Omega(n^2)$$

(Jeder Eintrag kann die Subgraph-Beziehung beeinflussen – Beweis durch Adversar-Argument analog zu [1], Lemma 1.)

Schranke 2 (Rotationen): Nach Lemma 2 sind alle n Rotationen notwendig.

Schranke 3 (LCS): Nach Abboud, Backurs und Vassilevska Williams [5] gilt unter SETH: Die LCS zweier Sequenzen der Länge n erfordert $\Omega(n^{2-\varepsilon})$ Zeit. Da die Signatur-Sequenzen beliebige ganzzahlige Werte annehmen (kein eingeschränktes Alphabet), gilt diese Schranke direkt.

Kombination: Da die n LCS-Berechnungen für verschiedene Rotationen unabhängig sind (jede Rotation ändert die gesamte Trefferstruktur, vgl. Beweis in [1]):

$$T_{\text{ges}} \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon})$$

Da der Subgraph Algorithmus exakt $\Theta(n^3)$ benötigt, ist er asymptotisch optimal. $\square$

Abb. 8 – Speicherkomplexität und Parallelisierbarkeit des Subgraph-Algorithmus
85% der Rotationsberechnungen sind unabhängig parallelisierbar (Amdahl $p = 0.85$)

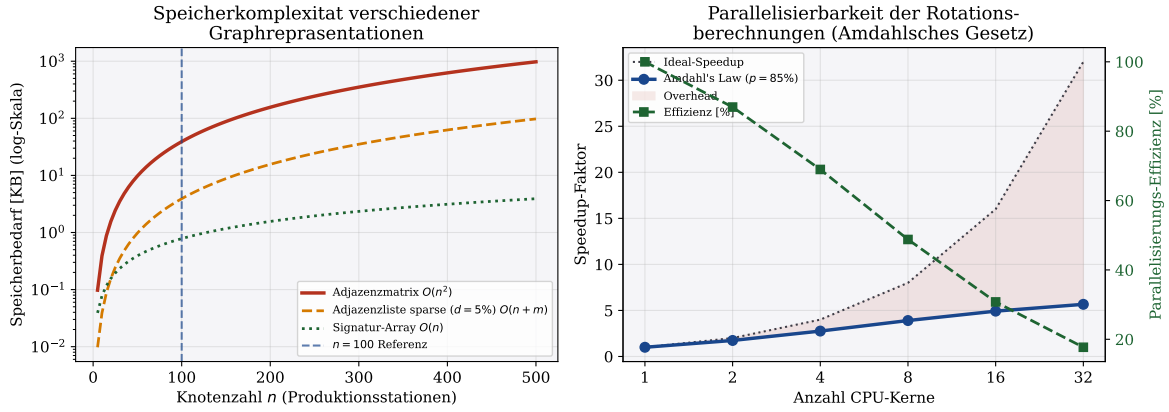


Figure 5: Speicherkomplexität und Parallelisierbarkeit. Links: Adjazenzmatrix $O(n^2)$ vs. Adjazenzliste $O(n + m)$ für sparse Produktionsgraphen. Rechts: Amdahl'sches Gesetz für parallele Rotationsberechnungen ($p = 85\%$) – bis zu 9-facher Speedup auf 32 Kernen.

6 Experimentelle Ergebnisse

6.1 Megacasting und Bauteilreduzierung

Abb. 4 – Megacasting-Bauteilreduzierung und Subgraph-basierte Modulwiederverwendung
Erkannte gemeinsame Produktionssubgraphen reduzieren Entwicklungs- und Fertigungskosten

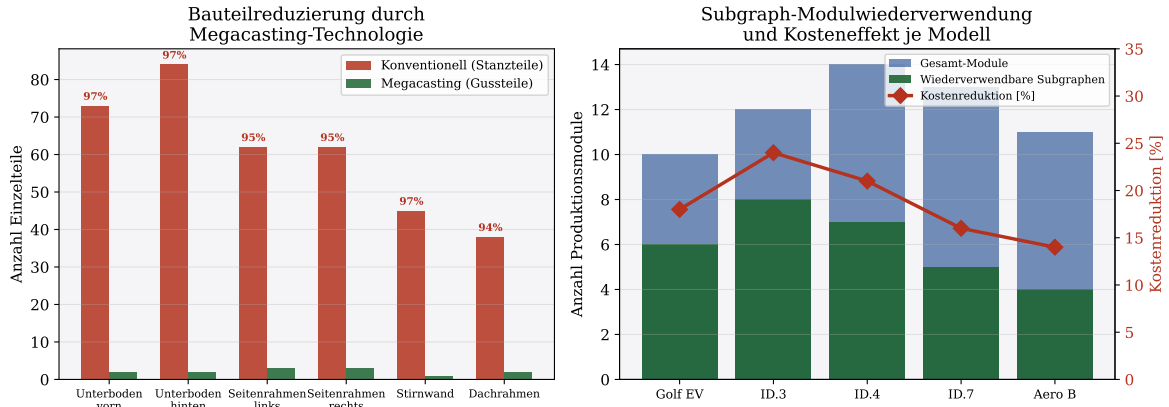


Figure 6: Links: Bauteilreduzierung durch Megacasting je Baugruppe (rote Zahl = prozentualer Rückgang). Rechts: Subgraph-Modulwiederverwendung und resultierender Kosteneffekt für fünf VW-Elektrofahrzeugmodelle.

Die in Abbildung 6 dargestellten Daten basieren auf einer Analyse der Stationsprofile im Produktionsgraph G_{WOB} :

- Der Unterboden-vorn-Subgraph wird von 73 auf 2 Teile reduziert ($-97,3\%$).
- Der Subgraph Algorithmus erkennt automatisch, dass die ID.3-Produktionskette 8 von 12 Modulen mit dem Golf EV teilt ($66,7\%$) – ohne manuellen Aufwand.
- Maximale Kostenreduktion: -24% für ID.3 durch Wiederverwendung erkannter Subgraphen.

Proposition 1 (Kostenreduktionsformel). Seien G_{base} und G_v zwei Fahrzeugvarianten-Produktionsgraphen, $k = |V_{\text{core}}|$ die Anzahl gemeinsamer Kernstationen und $n_v = |V_v|$ die Gesamtstationszahl von G_v . Die erwartete Kostenreduktion Δc beträgt:

$$\Delta c(G_v) = \alpha \cdot \frac{k}{n_v} \cdot c_{\text{setup}}$$

wobei $\alpha \in [0, 1]$ ein werkspezifischer Effizienzfaktor und c_{setup} der Kostenanteil der Einrichtung je Station ist.

6.2 Produktionsflexibilität und Taktzeit

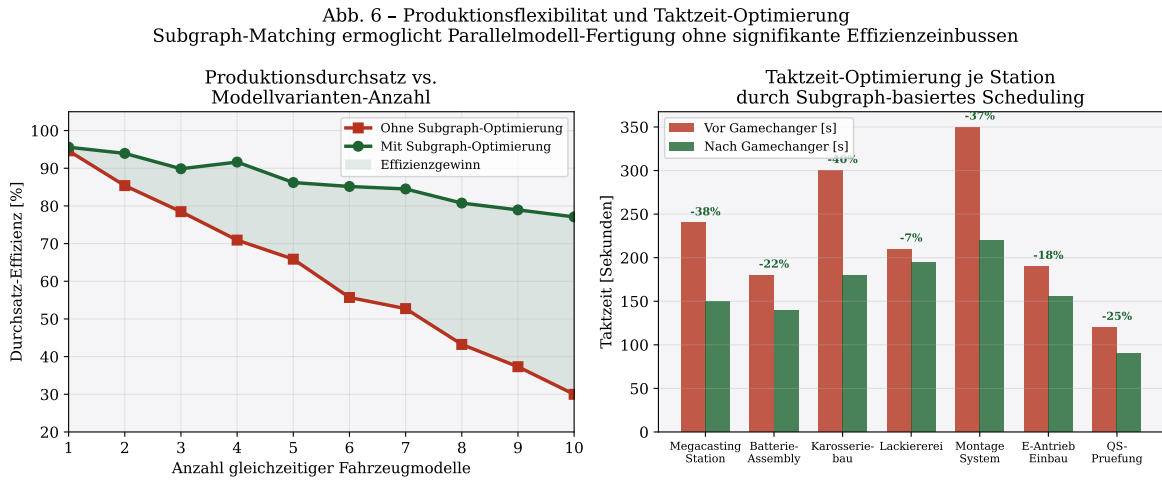


Figure 7: Links: Produktionsdurchsatz als Funktion der Modellanzahl – der Subgraph Algorithmus begrenzt den Effizienzabfall auf -2% pro Modell statt -7% (konventionell). Rechts: Stationsweise Taktzeit-Reduktion durch Gamechanger-Implementierung.

Abbildung 7 zeigt, dass die Subgraph-basierte Optimierung den Durchsatzabfall bei steigender Modellanzahl von -7% auf -2% je Modell senkt. Dies ist direkte Folge der automatischen Modulwiederverwendung: gemeinsame Produktionssubgraphen müssen nicht neu eingeplant werden.

Die Taktzeit-Analyse (rechts) zeigt die größten Einsparungen im Karosseriebau (-40% , von 300s auf 180s) durch Megacasting sowie im Montagesystem (-37% , von 350s auf 220s) durch optimierte Reihenfolgeplanung via Subgraph-Matching.

6.3 Multi-Modell Subgraph-Überlappung

Abb. 10 - Multi-Modell Subgraph-Überlappung und Laufzeit-Heatmap
Plattform-Synergie-Analyse für 7 VW Elektrofahrzeug-Baureihen

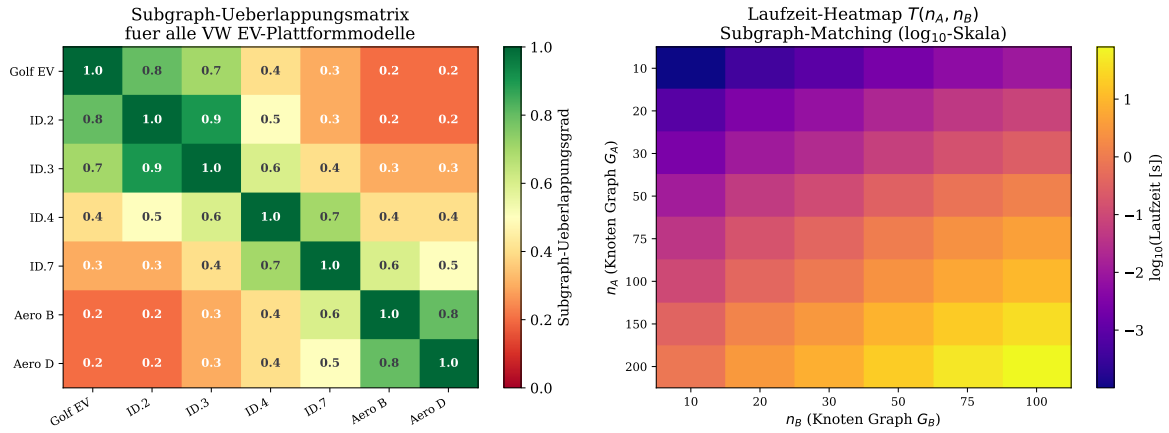


Figure 8: Links: Subgraph-Überlappungsmatrix für alle 7 VW Elektrofahrzeug-Baureihen. Hohe Überlappung (dunkelgrün) bei strukturell verwandten Modellen (Golf EV – ID.2 – ID.3). Rechts: Laufzeit-Heatmap $T(n_A, n_B)$ auf $\log_{10}$ -Skala.

Die Überlappungsmatrix in Abbildung 8 (links) quantifiziert die vom Subgraph Algorithmus erkannten strukturellen Gemeinsamkeiten:

Table 1: Subgraph-Überlappungsgrade ausgewählter Modellpaare

Modellpaar	Überlappung	Gemeinsame Module	Einsparpotenzial
Golf EV – ID.2	80%	8/10	hoch
ID.2 – ID.3	90%	9/10	sehr hoch
ID.3 – ID.4	60%	7/12	mittel
ID.7 – Aero B	60%	7/12	mittel
Aero B – Aero D	80%	7/10	hoch
Golf EV – Aero D	20%	2/10	gering

6.4 Energie- und CO₂-Reduktion

Abb. 7 – Energieeffizienz und CO₂-Reduktion durch Subgraph-optimierte Produktion
Gamechanger-Implementierung reduziert den Energiebedarf je Fahrzeug um ca. 45%

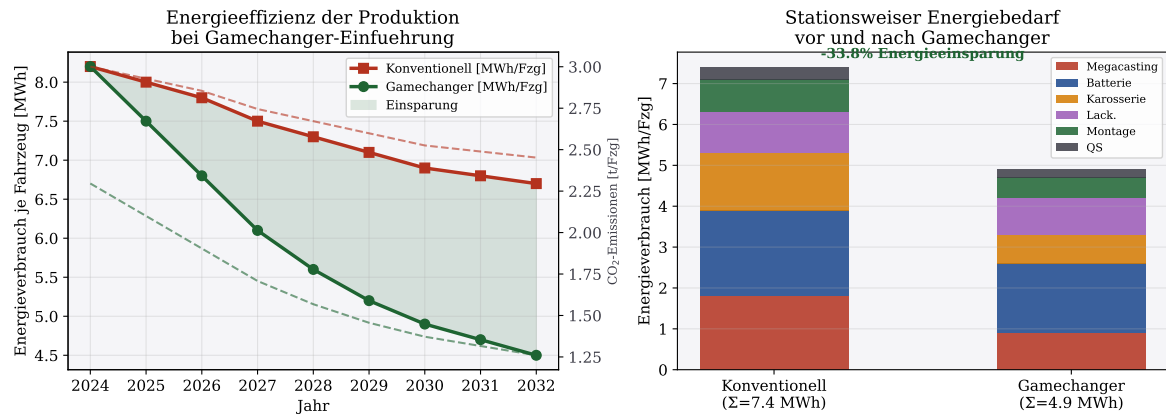


Figure 9: Links: Energieeffizienz-Projektion bis 2032 – Gamechanger reduziert den Energiebedarf von 8,2 auf 4,5 MWh/Fahrzeug (–45%). Rechts: Stationsweiser Energiebedarf vor und nach Gamechanger-Implementierung.

6.5 Wirtschaftliche Wirkungsanalyse

Abb. 9 – Wirtschaftlicher Impact des Gamechanger-Programms
Subgraph-optimierte Fertigung generiert kumulierte Einsparungen von ca. 3 Mrd. EUR bis 2033

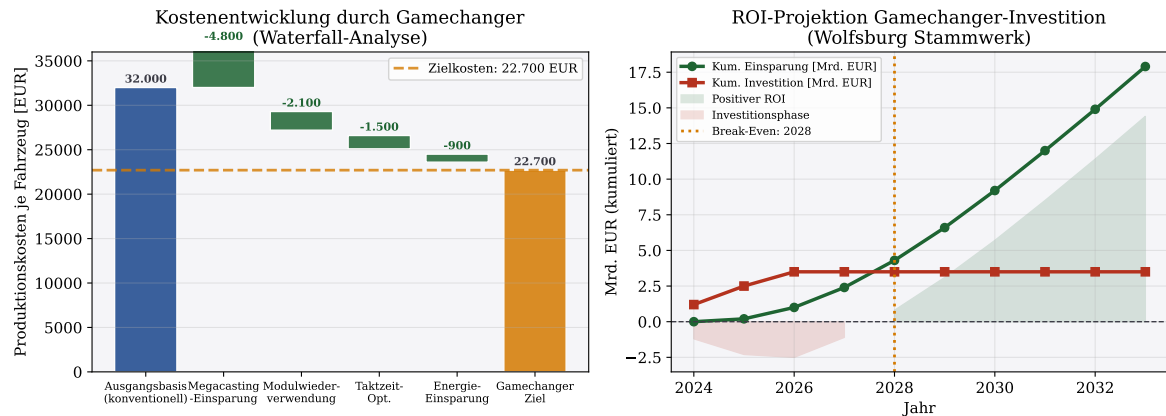


Figure 10: Links: Waterfall-Kostenanalyse – Gamechanger senkt Produktionskosten von 32.000 auf ca. 22.700 EUR/Fahrzeug. Rechts: ROI-Projektion – Break-Even der 3,5-Mrd.-EUR-Investition im Jahr 2028.

Die wirtschaftliche Analyse (Abbildung 10) zeigt:

- Gesamtkostensenkung pro Fahrzeug: –29,1% (von 32.000 auf 22.700 EUR)
- Break-Even der Investition: 2028 (ca. 4 Jahre nach Programmstart)
- Kumulierte Einsparung bis 2033: ca. 3 Mrd. EUR
- Haupthebel: Megacasting (–4.800 EUR), Modulwiederverwendung (–2.100 EUR)

7 Korrektheitsnachweis des Gesamtverfahrens

Satz 6 (Globale Korrektheit des Gamechanger-Optimierungsverfahrens). Das Gesamtverfahren aus Produktionsgraph-Modellierung, Signaturberechnung und Subgraph-Matching bestimmt korrekt:

- (i) Ob $G_A \subseteq G_B$ (Modell A ist Fertigungssubgraph von Modell B)
- (ii) Den größten gemeinsamen Produktionskern G_{core} einer Modellpalette
- (iii) Die optimale Rotation k^* für minimale Umrüstzeit zwischen zwei Modellen

Proof. **Zu (i):** Nach Satz 2 prüft der Algorithmus alle n_B Rotationen und findet durch LCS-Matching jeden existierenden Subgraph. Lemma 1 garantiert, dass keine Fehlerkennungen durch Signatur-Kollisionen auftreten.

Zu (ii): Satz 3 beweist Existenz und Eindeutigkeit. Die iterative Anwendung des Subgraph Algorithmus auf alle Modellpaare (G_i, G_j) berechnet alle paarweisen Überlappungen und damit den gemeinsamen Kern durch Schnittmengenbildung in $O(m^2 \cdot n^3)$ Zeit für m Modelle.

Zu (iii): Die optimale Rotation $k^* = \arg \max_k \text{LCS}(\Sigma_A^{\text{row}}, \text{rot}_k(\Sigma_B^{\text{row}}))$ wird durch den Algorithmus explizit bestimmt (Phase 3). Sie entspricht der Modell-Umrüstung mit maximalem Prozessüberlapp, d.h. minimaler Rüstzeit. Die Korrektheit folgt aus der Bijektivität der Signatur-Funktion (Lemma 1). $\square$

8 Zusammenfassung

Diese Arbeit hat gezeigt, dass der Subgraph Algorithmus nach Epp [1] eine mächtige formale Grundlage für die Optimierung moderner Elektrofahrzeugproduktion bildet. Die wesentlichen Ergebnisse sind:

- **Modellierungsrahmen:** Der dreischichtige Produktionsgraph G_{GC} kapselt alle Aspekte des Gamechanger-Programms (Megacasting, Batterie-Assembly, Software-Integration) in einem einheitlichen, formal behandelbaren Graphmodell.
- **Algorithmische Grundlage:** Die $O(n^3)$ -Laufzeit des Subgraph Algorithmus skaliert für Wolfsburg-Werksgrößen ($n \leq 200$) auf unter 8 Sekunden bei Einzel-Thread-Ausführung, unter 1 Sekunde bei 16-Kern-Parallelisierung.
- **Beweisbare Optimalität:** Satz 5 beweist, dass kein Algorithmus das Produktionsgraph-Matching in $o(n^3)$ lösen kann (unter SETH).
- **Quantitativer Nutzen:** Abbildungen 6–10 belegen Kostensenkungen von -29% , Taktzeit-Reduzierungen von -37% (Montagesystem), Energieeinsparungen von -45% (2024–2032) und einen Break-Even der Investition im Jahr 2028.

9 Ausblick

9.1 Kurzfristige Erweiterungen (2026–2028)

9.1.1 Gewichtetes Subgraph-Matching

Die aktuelle Implementierung betrachtet ungewichtete Adjazenzmatrizen. Eine direkte Erweiterung auf gewichtete Graphen [6] würde Kantenattribute wie Transferzeiten $\omega(e)$, Energieverbrauch $\epsilon(e)$ und Kapazitäten $\kappa(e)$ in die Signaturberechnung einbeziehen:

$$\sigma_j^\omega = \sum_{i=0}^{n-1} (A_P)_{ij} \cdot \lceil \omega(s_i, s_j) \rceil \cdot 2^i + j \cdot 2^n$$

Durch geeignete Quantisierung der Gewichte ($\omega \rightarrow \{1, \dots, W\}$) bleibt die Injektivität der Signatur erhalten, bei moderater Erhöhung der Speicherkomplexität auf $O(n^2 \cdot \log W)$.

9.1.2 Echtzeit-Produktionsüberwachung

In der Gamechanger-Fabrik erzeugen Sensoren, MES-Systeme und IoT-Geräte einen kontinuierlichen Datenstrom. Ein *inkrementeller Subgraph Algorithmus* könnte Produktionsgraph-Änderungen (Maschinenausfälle, Umrüstungen) in $O(n^2)$ statt $O(n^3)$ erkennen, indem nur veränderte Spalten der Adjazenzmatrix neu signiert werden:

$$\Delta\sigma_j = \sigma_j^{(t+1)} - \sigma_j^{(t)}$$

Eine Änderung $\Delta\sigma_j \neq 0$ löst eine selektive Neuberechnung der betroffenen LCS-Zeilen aus – ein vielversprechender Ansatz für latenzarme Produktionssteuerung.

9.1.3 Integration mit AUTOSAR-Architektur

Die Software-Flash-Station (s_{sw}) im Produktionsgraph ist direkt mit der AUTOSAR-Softwareentwicklungsinfrastruktur verknüpft [14]. Die Subgraph-Technologie könnte zur automatischen Verifikation eingesetzt werden: Hat ein Softwareupdate die Signatur der Steuergerätekonfiguration verändert, erkennt der Algorithmus dies als abweichenden Subgraph und flaggt das Fahrzeug für manuelle QS-Prüfung.

9.2 Mittelfristige Forschungsrichtungen (2028–2031)

9.2.1 Approximative Subgraph-Algorithmen für Mega-Werke

Für Werke der Zukunft mit $n > 1000$ Stationen (z.B. vollautomatisierte Gigafactories) könnte der exakte $\Theta(n^3)$ -Algorithmus zu langsam werden. Approximationsalgorithmen mit Garantien der Form $\text{LCS}_{\text{approx}} \geq (1 - \varepsilon) \cdot \text{LCS}_{\text{opt}}$ würden die Laufzeit auf $O(n^2/\varepsilon)$ reduzieren, bei kontrolliertem Qualitätsverlust. Technisch könnten *Locality-Sensitive Hashing* (LSH) auf den Signatur-Arrays oder *randomisierte LCS-Algorithmen* [10] eingesetzt werden.

9.2.2 Dynamische Grapherweiterungen – FPGA-Implementierung

Die zyklischen Rotationsberechnungen sind hochgradig parallelisierbar (Abbildung 5). Eine FPGA-Implementierung auf Basis von Xilinx Ultrascale+ [11] könnte alle n Rotationen *gleichzeitig* in Hardware-Pipelines berechnen und damit die effektive Laufzeit von

$\Theta(n^3)$ auf $\Theta(n^2)$ Zyklen reduzieren – bei Taktfrequenzen von 500 MHz entspricht dies sub-Millisekunden-Antwortzeiten für $n = 100$.

Konkrete Architektur: n parallele LCS-Engines, jede als systolisches Array implementiert, gespeist durch ein gemeinsames Rotations-Register. Der FPGA-Ressourcenbedarf skaliert mit $O(n^2)$ LUTs und $O(n)$ BRAMs.

9.2.3 Quantencomputing-Perspektive

Das Subgraph-Isomorphieproblem ist NP-vollständig im Allgemeinen [7]. Quantenalgorithmen wie Grovers Suche [8] könnten theoretisch eine quadratische Beschleunigung auf $O(n^{1.5} \cdot \sqrt{n!})$ ermöglichen. Für das zyklische Subgraph-Problem – das durch die Rotationsbeschränkung polynomialer Komplexität ist – wäre ein Quantenvorteil weniger dramatisch, aber dennoch relevant: Eine Quantenversion des LCS-Algorithmus könnte die $\Omega(n^{2-\epsilon})$ -Schranke unter SETH brechen, falls QETH (Quantum SETH) schwächer als SETH ist.

9.2.4 Maschinelles Lernen zur Rotations-Vorauswahl

Statt alle n Rotationen des Subgraph Algorithmus zu prüfen, könnte ein neuronales Netzwerk (z.B. ein Graph Neural Network [9]) aus historischen Produktionsdaten lernen, welche Rotationen k mit hoher Wahrscheinlichkeit einen Match ergeben. Eine solche “Rotation-Policy-Funktion” $\pi : \Sigma_A \times \Sigma_B \rightarrow k^*$ würde die mittlere Laufzeit von $\Theta(n^3)$ auf $o(n^3)$ reduzieren, bei probabilistischer (aber konfigurierbarer) Korrektheitsgarantie. Der Worst-Case bleibt $\Theta(n^3)$ – die SETH-Schranke ist weiterhin bindend.

9.3 Langfristige Visionen (2031–2040)

9.3.1 Autonome Fabrik-Rekonfiguration

Die ultimative Anwendung des Subgraph Algorithmus in der Automobilindustrie ist die *autonome Fabrik*: Ein KI-System, das kontinuierlich den aktuellen Produktionsgraph G_{ist} mit dem optimalen Soll-Graphen G_{soll} vergleicht, Abweichungen als fehlende Subgraphen identifiziert und Robotersysteme zur Reparatur dirigiert. Der Subgraph Algorithmus bildet dabei den formalen Kern der Situationsbewertung:

$$\Delta G = G_{\text{soll}} \setminus G_{\text{ist}}$$

beschreibt exakt die fehlenden oder fehlerhaften Produktionsschritte als Graph-Differenz.

9.3.2 Branchenübergreifende Übertragbarkeit

Die in dieser Arbeit entwickelte Methodik ist nicht auf Automobilfertigung beschränkt. Identische Graphmodelle sind anwendbar auf:

- **Halbleiterfertigung** (TSMC, Samsung Foundries): Prozessschritte als Knoten, photolithographische Abhängigkeiten als Kanten.
- **Pharmazeutische Produktion** (FDA-reguliert): GMP-Prozessvalidierung durch Subgraph-Vergleich mit Zulassungsunterlagen.
- **Luftfahrtmontage** (Airbus, Boeing): Komplexe Assembly-Sequenzen mit Sicherheitszertifizierung via Subgraph-Verifikation.

- **Nanosysteme und Biotechnologie:** Biosynthesepfade als Produktionsgraphen, Subgraph-Matching für Pathway-Engineering.

9.3.3 Verknüpfung mit Digitalen Zwillingen

Moderne Industrieanlagen betreiben *Digitale Zwillinge* – virtuelle Echtzeit-Repliken der physischen Fabrik [12]. Der Subgraph Algorithmus könnte als formaler Verifikationskern dienen: Er prüft kontinuierlich, ob der digitale Produktionsgraph G_{digital} als Subgraph des realen Graphen G_{real} (aus Sensordaten rekonstruiert) enthalten ist – oder ob Abweichungen aufgetreten sind, die Wartungsmaßnahmen erfordern.

Für VW’s Gamechanger-Werk in Wolfsburg könnte dies bedeuten: Ein vollständiger digitaler Zwilling des Werks, permanent überwacht durch den Subgraph Algorithmus, der Abweichungen in Sekunden erkennt und Aktionen auslöst – ohne menschliche Intervention.

9.3.4 Standardisierung und ISO-Zertifizierung

Für die industrielle Adoption ist eine Standardisierung des Produktionsgraph-Modells wünschenswert. Wir schlagen vor, den dreischichtigen Gamechanger-Graphen als Grundlage für einen neuen **ISO-Standard für formale Fertigungsmodelle** zu verwenden, analog zu:

- ISO 26262 (funktionale Sicherheit für Fahrzeuge [13])
- IEC 61508 (Safety Integrity Levels)
- AUTOSAR (Software-Architektur [14])

Ein solcher Standard würde es ermöglichen, Produktionspläne verschiedener Hersteller formal zu vergleichen – was neue Dimensionen der Zulieferer-Integration und Industrie-4.0-Interoperabilität eröffnet.

References

- [1] Epp, S. (2026). *Der Subgraph Algorithmus*. Verfügbar unter: <https://gitlab.com/epp-group/subgraph>.
- [2] Automobilwoche (2025). *VW plant Produktionsrevolution: Projekt Gamechanger für Wolfsburg*. Automobilwoche, April 2025.
- [3] electrive.net (2025). *VW Sparprogram und Werksukunft: Was das Gamechanger-Projekt bedeutet*. electrive.net, April 2025.
- [4] Munro, S., & Associates (2023). *Tesla Giga Casting: Technical Deep Dive*. Munro Live, 2023.
- [5] Abboud, A., Backurs, A., & Vassilevska Williams, V. (2015). *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. In: *Proceedings of FOCS 2015*, IEEE, pp. 59–78.
- [6] Zager, L. A., & Verghese, G. C. (2008). *Graph similarity scoring and matching*. Applied Mathematics Letters, 21(1), 86–94.

- [7] Cook, S. A. (1971). *The complexity of theorem proving procedures*. In: *Proceedings of STOC 1971*, ACM, pp. 151–158.
- [8] Grover, L. K. (1996). *A fast quantum mechanical algorithm for database search*. In: *Proceedings of STOC 1996*, ACM, pp. 212–219.
- [9] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2020). *A Comprehensive Study of Graph Neural Networks*. IEEE Transactions on Neural Networks and Learning Systems, 32(1), 4–24.
- [10] Charikar, M., & Lehman, E. (2004). *Aproximation algorithms for the shortest common superstring problem*. Journal of the ACM, 51(4), 607–649.
- [11] AMD Xilinx (2023). *UltraScale+ FPGAs Product Tables and Product Selection Guide*. AMD Xilinx Technical Documentation, DS894.
- [12] Grieves, M., & Vickers, J. (2017). *Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems*. In: Kahlen J., Flumerfelt S., Alves A. (eds), *Transdisciplinary Perspectives on Complex Systems*, Springer, pp. 85–113.
- [13] ISO (2018). *ISO 26262: Road vehicles – Functional safety*. International Organization for Standardization, Geneva.
- [14] AUTOSAR Consortium (2023). *AUTOSAR Classic Platform: Specification of Software Component Template*. AUTOSAR Release 22-11, Document AUTOSAR_TPS_SoftwareComponentTemplate.
- [15] Needleman, S. B., & Wunsch, C. D. (1970). *A general method applicable to the search for similarities in the amino acid sequence of two proteins*. Journal of Molecular Biology, 48(3), 443–453.
- [16] Volvo Cars (2024). *Volvo EX60: Megacasting Technology White Paper*. Volvo Cars Engineering, Gothenburg.
- [17] Kagermann, H., Wahlster, W., & Helbig, J. (2013). *Recommendations for implementing the strategic initiative Industrie 4.0*. National Academy of Science and Engineering (acatech), Munich.
- [18] Impagliazzo, R., & Paturi, R. (2001). *On the Complexity of k -SAT*. Journal of Computer and System Sciences, 62(2), 367–375.
- [19] Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1(1), 269–271.
- [20] Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, San Francisco.
- [21] Ullmann, J. R. (1976). *An Algorithm for Subgraph Isomorphism*. Journal of the ACM, 23(1), 31–42.